1 . DDL STATEMENTS:

1. users:

```
CREATE TABLE Users (
user_id INT PRIMARY KEY AUTO_INCREMENT,
name VARCHAR(100) NOT NULL,
email VARCHAR(150) UNIQUE NOT NULL,
role VARCHAR(50) NOT NULL
);
```

2. security_analysts:

```
CREATE TABLE Security_Analysts (
analyst_id INT PRIMARY KEY AUTO_INCREMENT,
name VARCHAR(100) NOT NULL,
email VARCHAR(150) UNIQUE NOT NULL,
specialization VARCHAR(100),
experience_years INT
);
```

3. incidents:

```
CREATE TABLE Incidents (
incident_id INT PRIMARY KEY AUTO_INCREMENT,
title VARCHAR(200) NOT NULL,
description TEXT,
severity VARCHAR(20),
status VARCHAR(30),
detected_at DATETIME,
reported_by INT,
assigned_to INT,
FOREIGN KEY (reported_by)
REFERENCES Users(user_id),
FOREIGN KEY (assigned_to)
REFERENCES Security_Analysts(analyst_id)
);
```

4. threat_categories :

```
CREATE TABLE Threat_Categories (
category_id INT PRIMARY KEY AUTO_INCREMENT,
category_name VARCHAR(100) NOT NULL
);
```

5. threats :

```
CREATE TABLE Threats (
threat_id INT PRIMARY KEY AUTO_INCREMENT,
threat_name VARCHAR(100) NOT NULL,
description TEXT,
threat_level VARCHAR(20)
);
```

6. assets :

```
CREATE TABLE Assets (
asset_id INT PRIMARY KEY AUTO_INCREMENT,
asset_name VARCHAR(150) NOT NULL,
asset_type VARCHAR(50),
ip_address VARCHAR(45),
operating_system VARCHAR(100),
owner VARCHAR(100)
);
```

7. incident_threats :

```
CREATE TABLE Incident_Threats (
incident_id INT,
threat_id INT,
PRIMARY KEY (incident_id, threat_id),
FOREIGN KEY (incident_id)
REFERENCES Incidents(incident_id),
FOREIGN KEY (threat_id)
REFERENCES Threats(threat_id)
);
```

8. incident_assets :

```
CREATE TABLE Incident_Assets (
incident_id INT,
asset_id INT,
PRIMARY KEY (incident_id, asset_id),
FOREIGN KEY (incident_id)
REFERENCES Incidents(incident_id),
FOREIGN KEY (asset_id)
REFERENCES Assets(asset_id)
);
```

9. evidence :

```
CREATE TABLE Evidence (
evidence_id INT PRIMARY KEY AUTO_INCREMENT,
incident_id INT NOT NULL,
evidence_type VARCHAR(50),
description TEXT,
evidence_value VARCHAR(500),
collected_at DATETIME,
FOREIGN KEY (incident_id)
REFERENCES Incidents(incident_id)
);
```

10. response_actions:

```
CREATE TABLE Response_Actions (
action_id INT PRIMARY KEY AUTO_INCREMENT,
incident_id INT NOT NULL,
performed_by INT,
action_description TEXT,
performed_at DATETIME,
FOREIGN KEY (incident_id)
REFERENCES Incidents(incident_id),
FOREIGN KEY (performed_by)
```

```
REFERENCES Users(user_id)
);
```

11. incident_comments :

```
CREATE TABLE Incident_Comments (
comment_id INT PRIMARY KEY AUTO_INCREMENT,
incident_id INT NOT NULL,
user_id INT NOT NULL,
comment TEXT NOT NULL,
created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
FOREIGN KEY (incident_id)
REFERENCES Incidents(incident_id),
FOREIGN KEY (user_id)
REFERENCES Users(user_id)
);
```

2. DML QUERIES :

A. INSERT Queries

Users:

INSERT INTO Users (name, email, role) VALUES ('Rahul Patil', 'rahul.patil@gmail.com', 'Employee'), ('Amit Shah', 'amit.shah@gmail.com', 'Employee'), ('Priya Joshi', 'priya.joshi@gmail.com', 'Employee'), ('Sneha Kulkarni', 'sneha.kulkarni@gmail.com', 'Employee'), ('Vikram Deshmukh', 'vikram.d@gmail.com', 'Employee');

Security Analysts:

INSERT INTO Security_Analysts (name, email, specialization, experience_years) VALUES ('Rahul Patil', 'rahul@company.com', 'Network Security', 3), ('Amit Shah', 'amit@company.com', 'Incident Response', 5), ('Priya Joshi', 'priya@company.com', 'Threat Analysis', 4);

Threat Categories:

INSERT INTO Threat_Categories (category_name) VALUES ('Malware'), ('Phishing'), ('Network Attack'), ('Insider Threat'), ('Data Breach');

Threats:

INSERT INTO Threats (threat_name, description, threat_level) VALUES ('Phishing', 'Fraudulent attempt to obtain sensitive information', 'HIGH'), ('Ransomware', 'Malware that encrypts files and demands payment', 'CRITICAL'), ('SQL Injection', 'Injection of malicious

SQL statements', 'HIGH'), ('DDoS', 'Distributed denial of service attack', 'HIGH'), ('Insider Threat', 'Threat originating from an authorized user', 'MEDIUM');

Assets:

INSERT INTO Assets (asset_name, asset_type, ip_address, operating_system, owner) VALUES ('Web Server 01', 'Server', '192.168.1.10', 'Ubuntu 22.04', 'IT Department'), ('Database Server 01', 'Database Server', '192.168.1.20', 'Ubuntu 22.04', 'Database Team'), ('Employee Laptop 01', 'Laptop', '192.168.1.101', 'Windows 11', 'HR Department'), ('Mail Server 01', 'Mail Server', '192.168.1.30', 'Ubuntu 20.04', 'IT Department'),

('Firewall 01', 'Network Device', '192.168.1.1', 'Cisco IOS', 'Network Team');

Incidents:

INSERT INTO Incidents (title, description, severity, status, detected_at, reported_by, assigned_to) VALUES ('Phishing Email Attack', 'Employee received a suspicious phishing email.', 'HIGH', 'OPEN', '2026-09-01 09:30:00', 1, 1), ('Ransomware Detection', 'Ransomware activity detected on an employee laptop.', 'CRITICAL', 'INVESTIGATING', '2026-09-02 11:15:00', 2, 2), ('SQL Injection Attempt', 'Possible SQL injection detected on web application.', 'HIGH', 'OPEN', '2026-09-03 14:20:00', 3, 3), ('DDoS Attack', 'Large amount of malicious traffic detected.',

'CRITICAL', 'RESOLVED', '2026-09-04 18:00:00', 4, 1), ('Unauthorized Login', 'Multiple unauthorized login attempts detected.', 'MEDIUM', 'OPEN', '2026-09-05 10:45:00', 5, 2);

Incident_Threats:

INSERT INTO Incident_Threats (incident_id, threat_id) VALUES (1, 1), (2, 2), (3, 3), (4, 4), (5, 5);

Incident_Assets:

INSERT INTO Incident_Assets (incident_id, asset_id) VALUES (1, 4), (2, 3), (3, 1), (4, 5), (5, 2);

Evidence:

INSERT INTO Evidence (incident_id, evidence_type, description, evidence_value, collected_at) VALUES (1, 'Email', 'Suspicious email received by employee', 'phishing_email_001.eml', '2026-09-01 10:00:00'), (2, 'File', 'Encrypted file discovered on laptop', 'ransomware_sample.exe', '2026-09-02 11:45:00'), (3, 'Log', 'Web server SQL injection logs', 'sql_error_log_001.txt', '2026-09-03 14:45:00'), (4, 'Network Traffic', 'Abnormal incoming traffic', 'traffic_capture_001.pcap', '2026-09-04 18:30:00');

Response_Actions:

INSERT INTO Response_Actions (incident_id, performed_by, action_description, performed_at) VALUES (1, 1, 'Blocked malicious email sender', '2026-09-01 11:00:00'), (2, 2, 'Isolated infected laptop from network', '2026-09-02 12:00:00'), (3, 3, 'Blocked suspicious SQL requests', '2026-09-03 15:00:00'), (4, 1, 'Blocked malicious IP addresses', '2026-09-04 19:00:00');

Incident_Comments:

INSERT INTO Incident_Comments (incident_id, user_id, comment) VALUES (1, 1, 'The email appears to be a phishing attempt.'),

(2, 2, 'The affected laptop has been isolated.'), (3, 3, 'SQL injection activity was confirmed.'), (4, 4, 'DDoS traffic has been blocked.');

B. SELECT — Critical incidents

```
SELECT incident_id, title, severity, status FROM Incidents WHERE severity = 'CRITICAL';
```

C. SELECT — Incidents with assigned analyst

```
SELECT i.incident_id, i.title, i.severity, a.name AS analyst FROM Incidents i JOIN
Security_Analysts a ON i.assigned_to = a.analyst_id;
```

D. NESTED QUERY — Phishing-related incidents

```
SELECT * FROM Incidents WHERE incident_id IN ( SELECT incident_id FROM
Incident_Threats WHERE threat_id = ( SELECT threat_id FROM Threats WHERE
threat_name = 'Phishing'
) );
```

E. UPDATE — Change incident status

```
UPDATE Incidents SET status = 'INVESTIGATING' WHERE incident_id = 1;
```

F. DELETE — Delete an incident

```
DELETE FROM Incidents WHERE incident_id = 10;
```

G. AGGREGATE — Count incidents by severity

```
SELECT severity, COUNT(*) AS total_incidents FROM Incidents GROUP BY severity;
```

H. AGGREGATE — Find threats affecting multiple incidents

```
SELECT t.threat_name, COUNT(it.incident_id) AS incident_count FROM Threats t JOIN Incident_Threats it ON t.threat_id = it.threat_id GROUP BY t.threat_id, t.threat_name HAVING COUNT(it.incident_id) > 1;
```

I — Incidents that are still open or being investigated

```
SELECT incident_id, title, severity, status, detected_at

FROM Incidents

WHERE status IN ('OPEN', 'INVESTIGATING')

ORDER BY detected_at DESC;
```

J — Assets affected by critical incidents

```
SELECT i.incident_id, i.title, i.severity, a.asset_name, a.ip_address

FROM Incidents i

JOIN Incident_Assets ia ON i.incident_id = ia.incident_id

JOIN Assets a ON ia.asset_id = a.asset_id

WHERE i.severity = 'CRITICAL'

ORDER BY i.detected_at DESC;
```

3. JOIN Queries :

These are especially useful because your database has several related tables.

JOIN — Incident and reporter

SELECT i.incident_id, i.title, u.name AS reporter, i.severity FROM Incidents i JOIN Users u ON i.reported_by = u.user_id;

JOIN — Incident and assigned analyst

SELECT i.incident_id, i.title, i.severity, a.name AS analyst FROM Incidents i JOIN Security_Analysts a ON i.assigned_to = a.analyst_id;

Concept: JOIN, table aliases

Multiple JOIN — Incident, threat and asset

SELECT i.incident_id, i.title, t.threat_name, a.asset_name FROM Incidents i JOIN Incident_Threats it ON i.incident_id = it.incident_id JOIN Threats t ON it.threat_id = t.threat_id JOIN Incident_Assets ia ON i.incident_id = ia.incident_id JOIN Assets a ON ia.asset_id = a.asset_id;

4) Procedure + Cursor — Generate Analyst Incident Report :

```
DELIMITER //
CREATE PROCEDURE Generate_Analyst_Report(IN p_analyst_id INT)
BEGIN
DECLARE done INT DEFAULT 0;
DECLARE v_incident_id INT;
DECLARE v_title VARCHAR(200);
DECLARE v_severity VARCHAR(20);
DECLARE v_status VARCHAR(30);
DECLARE incident_cursor CURSOR FOR
SELECT incident_id, title, severity, status
FROM Incidents
WHERE assigned_to = p_analyst_id;
DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = 1;
OPEN incident_cursor;
report_loop: LOOP
FETCH incident_cursor
INTO v_incident_id, v_title, v_severity, v_status;
IF done = 1 THEN
LEAVE report_loop;
END IF;
SELECT
v_incident_id AS Incident_ID,
v_title AS Title,
v_severity AS Severity,
```

```
v_status AS Status;
END LOOP;
CLOSE incident_cursor;
END //
DELIMITER ;
```

5. Function — Calculate Incident Count:

```
DELIMITER //
CREATE FUNCTION Get_Analyst_Incident_Count(p_analyst_id INT)
RETURNS INT
DETERMINISTIC
BEGIN
DECLARE incident_count INT;
SELECT COUNT(*)
INTO incident_count
FROM Incidents
WHERE assigned_to = p_analyst_id;
RETURN incident_count;
END //
DELIMITER ;

EXECUTE => SELECT Get_Analyst_Incident_Count(1) AS Total_Incidents;
```

6. Trigger — Automatic Status Timestamp :

For this trigger, first add the resolved_at column to Incidents.

Step 1 — Add column

```
ALTER TABLE Incidents ADD COLUMN resolved_at DATETIME;
```

Step 2 — Create trigger

```
DELIMITER //
 CREATE TRIGGER Incident_Status_Update BEFORE UPDATE ON Incidents
 FOR EACH ROW
 BEGIN
 IF NEW.status IN ('RESOLVED', 'CLOSED')
        AND OLD.status NOT IN ('RESOLVED', 'CLOSED') THEN SET NEW.resolved_at =
CURRENT_TIMESTAMP;
END IF;
 END //
 DELIMITER ;
```

Test it:

```
UPDATE Incidents SET status = 'RESOLVED' WHERE incident_id = 1;
```

7) Function 2 — Calculate Incident Risk Score :

```
 DELIMITER //
CREATE FUNCTION Calculate_Risk_Score( p_severity VARCHAR(20), p_threat_level
VARCHAR(20) )
RETURNS INT
 DETERMINISTIC
 BEGIN
        DECLARE severity_score INT DEFAULT 1;
        DECLARE threat_score INT DEFAULT 1;
        SET severity_score = CASE UPPER(p_severity)
                WHEN 'LOW' THEN 1
                WHEN 'MEDIUM' THEN 2
                WHEN 'HIGH' THEN 3
                WHEN 'CRITICAL' THEN 4
                ELSE 1 END;
        SET threat_score = CASE UPPER(p_threat_level)
                WHEN 'LOW' THEN 1
                 WHEN 'MEDIUM' THEN 2
                 WHEN 'HIGH' THEN 3
                WHEN 'CRITICAL' THEN 4
                ELSE 1
                END;
        RETURN severity_score + threat_score;
        END //
DELIMITER ;

EXECUTION => SELECT Calculate_Risk_Score('LOW', 'LOW') AS Risk_Score;
```